

Object-Oriented Programming

Final Project Report

Homoerectus

Aulia Febriyanti

Chalimatus Sa'Diyyah

Najma Khonsa Tsabita

Rega Saputra

BooQ

Meeting Room & Seat Booking App

DAFTAR ISI

DAFTAR ISI.....	2
DAFTAR GAMBAR.....	3
BAB 1 PENDAHULUAN	4
1.1 Latar Belakang	4
1.2 Tujuan	4
1.3 Manfaat	4
1.4 Luaran Proyek.....	4
BAB II KONSEP	5
2.1 Class, Object, dan Package	5
2.2 Variable dan Methods	5
2.3 Inhenritance.....	6
2.4 Interface	6
2.5 Abstract Class	6
2.6 Encapsulation.....	6
2.7 Polymorphism	7
2.8 Access Modifier	7
BAB III HASIL	8
3.1 Application Overview	8
3.2 Code Structure	10
3.3 Fungsi Aplikasi	11
3.4 Database.....	12
3.5 OOP Implementation	12
BAB IV KESIMPULAN.....	23
4.1 Ringkasan Proyek	23
4.2 Hasil yang Dicapai.....	23
4.3 Kesimpulan Akhir.....	24

DAFTAR GAMBAR

Gambar 3. 1 Validasi Format.....	9
Gambar 3. 2 Validasi Waktu Booking.....	9
Gambar 3. 3 Booking Meeting Room.....	9
Gambar 3. 4 Booking Duplicate	9
Gambar 3. 5 Code Structure.....	10
Gambar 3. 6 Database	12
Gambar 3. 7 Class	13
Gambar 3. 8 Membuat Object Scanner dan Employee.....	13
Gambar 3. 9 Package	13
Gambar 3. 10 Variable ObjekPinjaman	14
Gambar 3. 11 Method Employee	14
Gambar 3. 12 ObjectPinjaman Class	15
Gambar 3. 13 MeetingRoom Class Inherit ObjectPinjaman	15
Gambar 3. 14 Public Class.....	16
Gambar 3. 15 Protected Variable.....	16
Gambar 3. 16 Encapsulation Employee Class	17
Gambar 3. 17 Polymorphism displayInfo()	18
Gambar 3. 18 Overriding displayInfo() Room	18
Gambar 3. 19 Overriding displayInfo() Seat	18
Gambar 3. 20 Overriding	18
Gambar 3. 21 Abstract class	19
Gambar 3. 22 Interface report.....	19
Gambar 3. 23 WeeklyReport Class.....	20
Gambar 3. 24 MonthlyReport Class	20
Gambar 3. 25 YearlyReport.....	21
Gambar 3. 26 DBConnection.....	22

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Proyek ini bertujuan untuk membangun sebuah aplikasi Booking Meeting Room dan Seat dengan menggunakan konsep Object-Oriented Programming (OOP) dalam bahasa pemrograman Java. Sistem ini dirancang untuk membantu karyawan dalam perusahaan melakukan pemesanan ruang meeting atau kursi dengan menggunakan aplikasi berbasis command-line interface (CLI). Tujuan utama adalah untuk memberikan kemudahan bagi pengguna dalam melakukan pemesanan serta mendapatkan laporan penggunaan secara terstruktur dan efisien.

1.2 Tujuan

Tujuan utama dari proyek ini adalah:

1. Mengimplementasikan sistem pemesanan ruang meeting dan kursi secara otomatis.
2. Menerapkan konsep-konsep dasar OOP seperti Inheritance, Polymorphism, Encapsulation, dan Interface.
3. Menghasilkan laporan bulanan dan tahunan yang dapat digunakan untuk memantau penggunaan ruang meeting dan kursi di perusahaan.

1.3 Manfaat

Manfaat dari proyek ini antara lain:

1. Meningkatkan efisiensi pemesanan ruang meeting dan kursi di perusahaan.
2. Mempermudah manajer atau administrator dalam memonitor dan mengelola pemesanan.
3. Memberikan laporan penggunaan yang berguna untuk analisis dan pengambilan keputusan.

1.4 Luaran Proyek

Luaran dari proyek ini adalah sebuah aplikasi berbasis CLI yang dapat digunakan oleh karyawan untuk:

1. Melakukan pemesanan ruang meeting dan kursi.
2. Mengedit, menghapus, dan melihat detail pemesanan.
3. Menghasilkan laporan penggunaan baik per bulan maupun per tahun.

BAB II KONSEP

Sistem ini dibangun dengan paradigma *Object-Oriented Programming* (OOP) menggunakan bahasa pemrograman Java. Pendekatan OOP memberikan struktur kode yang lebih rapi, modular, dan mudah dikembangkan.

Adapun konsep-konsep OOP yang diterapkan dalam proyek ini meliputi:

2.1 Class, Object, dan Package

Class sebagai dasar dari konsep OOP, class bertindak sebagai blueprint atau template yang mendefinisikan atribut dan perilaku objek-objek yang dihasilkan dari class tersebut. Pada aplikasi ini, beberapa class utama seperti Employee, MeetingRoom, dan Seat berfungsi sebagai entitas yang menggambarkan data terkait ruang rapat, kursi, dan karyawan.

Objek adalah instansi yang dihasilkan dari class. Misalnya, setiap kali kita membuat entitas seperti Employee dengan data spesifik, itu menjadi objek yang berisi data sesuai dengan class yang mendefinisikannya.

Package untuk mempermudah pengelolaan dan pengorganisasian kode, proyek ini memanfaatkan beberapa package untuk memisahkan bagian-bagian aplikasi. Paket-paket utama yang ada dalam proyek ini antara lain:

1. package_model: Berisi class yang mendefinisikan model objek seperti Employee, MeetingRoom, dan Seat.
2. package_transaction: Berisi logika transaksi pemesanan seperti Booking.
3. package_report: Berisi class untuk menghasilkan laporan seperti MonthlyReport dan WeeklyReport.

2.2 Variable dan Methods

Dalam OOP, variabel digunakan untuk menyimpan data. Pada aplikasi ini, variabel-variabel dideklarasikan di dalam class untuk menyimpan informasi terkait objek. Sebagai contoh: Pada class Employee, terdapat variabel seperti employeeId, employeeName, dan employeeDepartment.

Pada class MeetingRoom, terdapat variabel meetingroomId, meetingroomName, meetingroomCapacity, dan lainnya yang digunakan untuk menyimpan informasi terkait ruang rapat.

Methods mendefinisikan perilaku atau fungsi yang bisa dilakukan oleh objek. Setiap objek yang diciptakan dari class dapat mengakses metode-metode tersebut untuk berinteraksi dengan data mereka. Sebagai contoh:

1. Method `addEmployee()` digunakan untuk menambahkan data karyawan baru ke dalam sistem.
2. Method `bookMeetingRoom()` digunakan untuk memproses pemesanan ruang rapat setelah memeriksa ketersediaannya.
3. Method `generateReport()` yang ada pada interface `Report` digunakan untuk menghasilkan laporan berdasarkan periode tertentu (bulanan atau mingguan).

2.3 Inheritance

Inheritance adalah konsep yang memungkinkan sebuah kelas (class) untuk mewarisi properti dan metode dari kelas lain. Pada proyek ini, kami menggunakan abstract class `ObjekPinjaman` yang digunakan sebagai superclass untuk kelas `MeetingRoom` dan `Seat`. Dengan demikian, baik `MeetingRoom` maupun `Seat` dapat mewarisi atribut dan metode yang ada pada `ObjekPinjaman` seperti `id`, `name`, dan `isAvailable`.

2.4 Interface

Interface di Java adalah kontrak yang mendefinisikan metode-metode yang harus diimplementasikan oleh kelas yang mengimplementasikannya. Di proyek ini, kami menggunakan interface `Report` yang memiliki metode `generateReport()`. Interface ini diimplementasikan oleh kelas `MonthlyReport` dan `YearlyReport`, yang masing-masing menghasilkan laporan bulanan dan tahunan berdasarkan data pemesanan yang ada..

2.5 Abstract Class

Abstract class digunakan untuk mendefinisikan struktur dasar dari kelas-kelas yang lebih spesifik. `ObjekPinjaman` adalah abstract class yang digunakan untuk mendefinisikan atribut umum yang dimiliki oleh objek yang dapat dipinjam, seperti `id`, `name`, dan status `isAvailable`. Kelas `MeetingRoom` dan `Seat` adalah subclass yang mengimplementasikan abstract class ini dengan menambahkan atribut dan perilaku spesifik masing-masing objek.

2.6 Encapsulation

Encapsulation adalah teknik dalam OOP yang membatasi akses ke atribut dan metode dalam kelas dengan menggunakan access modifiers. Di proyek ini, atribut-atribut seperti `id`,

name, dan isAvailable di kelas MeetingRoom, Seat, dan ObjekPinjaman bersifat private, sehingga hanya bisa diakses melalui getter dan setter yang didefinisikan secara eksplisit. Dengan demikian, kami menjaga integritas data dan memastikan bahwa data hanya dapat dimodifikasi melalui cara yang terkontrol.

2.7 Polymorphism

Polymorphism memungkinkan objek untuk memiliki banyak bentuk. Dalam proyek ini, kami menerapkan polymorphism dengan menggunakan method overriding di kelas-kelas laporan seperti MonthlyReport dan YearlyReport. Meskipun metode generateReport() memiliki nama yang sama, setiap kelas memberikan implementasi yang berbeda sesuai dengan jenis laporannya, baik itu laporan bulanan atau tahunan.

Selain itu, polymorphism juga dapat diterapkan melalui method overloading, di mana sebuah kelas memiliki beberapa metode dengan nama yang sama, tetapi dengan parameter yang berbeda. Misalnya, konstruktor Employee yang dapat menerima parameter berbeda: satu dengan informasi lengkap seperti ID, nama, dan departemen, dan satu lagi hanya dengan ID dan nama, di mana departemen akan diberi nilai default.

2.8 Access Modifier

Access modifiers di Java digunakan untuk menentukan visibilitas atau akses terhadap atribut dan metode. Di proyek ini, kami menggunakan private, public, dan protected sesuai dengan kebutuhan. Misalnya, atribut id, name, dan isAvailable di kelas MeetingRoom dan Seat dideklarasikan sebagai private, sehingga hanya bisa diakses melalui metode getter dan setter yang bersifat public.

BAB III

HASIL

3.1 Application Overview

Aplikasi ini memungkinkan pengguna untuk melakukan pemesanan ruang meeting dan kursi. Fitur-fitur yang ada di antaranya adalah:

1. Pemesanan ruang meeting dan kursi.

Memungkinkan pengguna untuk memilih tanggal dan waktu yang diinginkan, serta memastikan ketersediaan ruangan atau kursi yang sesuai dengan permintaan mereka

2. Laporan penggunaan yang dapat dilihat per bulan dan per tahun.

Aplikasi ini juga dilengkapi dengan fitur laporan yang memungkinkan pengguna untuk melihat penggunaan ruang meeting dan kursi. Laporan ini dapat dilihat berdasarkan periode tertentu, baik itu per bulan maupun per tahun, sehingga memudahkan pengelola untuk memonitor dan menganalisis penggunaan fasilitas.

3. Kemampuan untuk menambah, mengedit, dan menghapus data ruang meeting dan kursi.

Pengguna yang memiliki hak akses dapat menambah, mengedit, atau menghapus data ruang meeting dan kursi. Fitur ini memungkinkan pengelola untuk memperbarui informasi terkait fasilitas yang ada dalam aplikasi sesuai kebutuhan.

Untuk memastikan aplikasi berfungsi dengan baik, beberapa validasi diterapkan pada proses pemesanan. Validasi ini bertujuan untuk mencegah terjadinya kesalahan input atau pemesanan yang tidak valid. Berikut beberapa jenis validasi yang diterapkan dalam aplikasi:

1. Validasi Tanggal dan Waktu

Agar pemesanan tidak dilakukan pada tanggal atau waktu yang sudah lewat, aplikasi memeriksa apakah tanggal yang dimasukkan oleh pengguna lebih besar atau sama dengan tanggal hari ini. Jika tanggal yang dimasukkan lebih kecil dari hari ini, sistem akan menampilkan pesan kesalahan yang mengatakan bahwa tanggal yang dimasukkan sudah lewat dan meminta pengguna untuk memasukkan tanggal yang valid.

```

Enter your choice: 8
Enter employee ID:
1
Enter meeting room ID:
1
Enter booking occasion:
Meeting
Enter booking date (yyyy-mm-dd):
abc
Enter start time (HH:mm):
08:00
Enter end time (HH:mm):
09:00
Invalid date or time format. Please enter in yyyy-MM-dd and HH:mm format.

```

Gambar 3. 1 Validasi Format

```

Enter employee ID:
1
Enter meeting room ID:
1
Enter booking occasion:
Meeting
Enter booking date (yyyy-mm-dd):
2025-02-02
Enter start time (HH:mm):
08:00
Enter end time (HH:mm):
09:00
The booking date and time cannot be in the past. Please choose a valid date and time.

```

Gambar 3. 2 Validasi Waktu Booking

2. Validasi Pemesanan Tumpang Tindih

Memeriksa ketersediaan ruang meeting atau kursi pada tanggal dan waktu yang dipilih. Jika ada pemesanan yang sudah ada pada waktu yang sama, aplikasi akan menampilkan pesan yang menyatakan bahwa fasilitas tersebut tidak tersedia pada waktu yang diminta.

```

Enter employee ID:
1
Enter meeting room ID:
1
Enter booking occasion:
Meeting
Enter booking date (yyyy-mm-dd):
2025-06-21
Enter start time (HH:mm):
08:00
Enter end time (HH:mm):
12:00
Booking created successfully with ID: 13

```

Gambar 3. 3 Booking Meeting Room

```

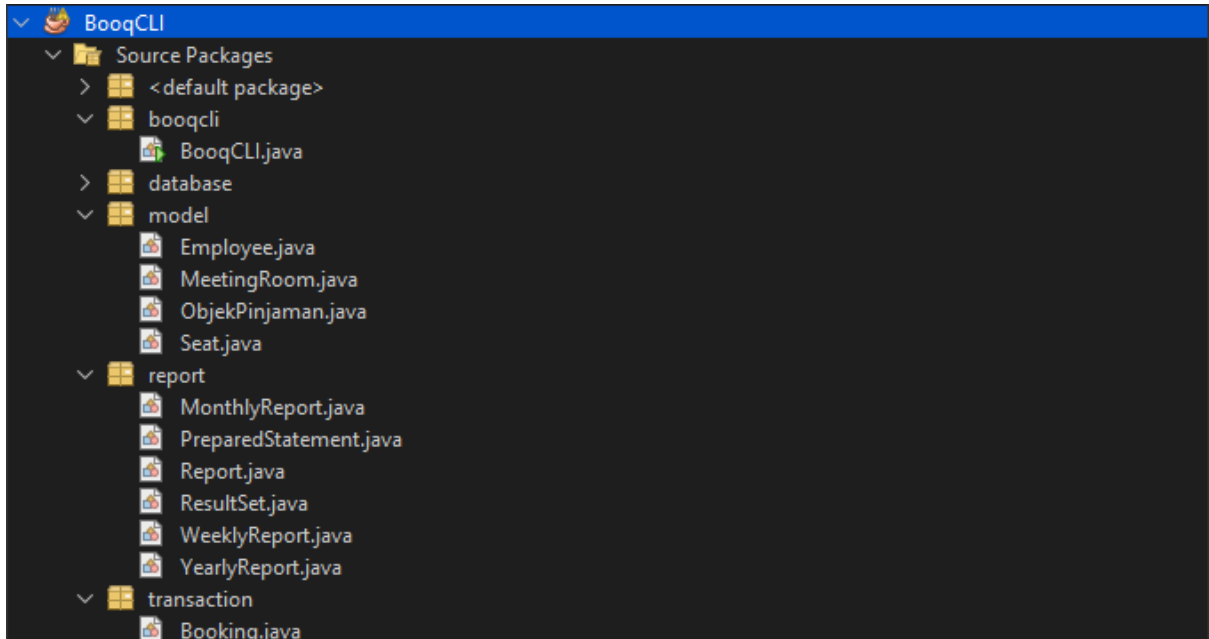
Enter your choice: 8
Enter employee ID:
1
Enter meeting room ID:
1
Enter booking occasion:
Training
Enter booking date (yyyy-mm-dd):
2025-06-21
Enter start time (HH:mm):
09:00
Enter end time (HH:mm):
11:30
The requested meeting room is not available during this time.

```

Gambar 3. 4 Booking Duplicate

Dengan adanya validasi ini, aplikasi dapat mencegah kesalahan pemesanan, meningkatkan akurasi data, dan memberikan pengalaman yang lebih baik bagi pengguna.

3.2 Code Structure



Gambar 3. 5 Code Structure

Kode aplikasi ini dibagi ke dalam beberapa paket sesuai dengan fungsinya:

1. package_model

Paket ini berisi kelas-kelas model yang mendeskripsikan objek-objek yang ada dalam aplikasi, seperti Employee, MeetingRoom, Seat, dan ObjekPinjaman. Setiap objek di dalam paket ini memiliki atribut dan metode untuk mendukung fungsi aplikasi, seperti menambah, mengedit, dan menghapus data.

2. package_transaction

Paket ini berisi logika yang menangani transaksi terkait pemesanan dan manajemen objek yang ada. Di dalamnya terdapat kelas Booking yang bertanggung jawab untuk membuat, mengedit, dan menghapus pemesanan ruang meeting dan kursi.

3. package_report

Paket ini berisi kelas untuk menangani pembuatan laporan. Misalnya, MonthlyReport dan YearlyReport digunakan untuk menghasilkan laporan berdasarkan waktu tertentu. Laporan-laporan ini digunakan oleh manajer atau administrator untuk memantau pemanfaatan ruang meeting dan kursi.

4. package_database

Paket ini berfungsi untuk mengelola koneksi ke database menggunakan DBConnection. Aplikasi ini menggunakan database MySQL untuk menyimpan dan mengambil data terkait ruang meeting, kursi, karyawan, dan pemesanan.

3.3 Fungsi Aplikasi

Berikut adalah daftar fungsi yang tersedia dalam aplikasi beserta deskripsi singkat tentang cara kerjanya:

1. Add Employee
Menambahkan data karyawan baru ke dalam sistem, mencakup employee_id, employee_name, dan employee_department.
2. Edit Employee
Memungkinkan pengguna untuk mengedit informasi karyawan yang sudah terdaftar, seperti nama atau departemen.
3. Delete Employee
Menghapus data karyawan berdasarkan employee_id yang diberikan.
4. See All Employees
Menampilkan daftar semua karyawan yang terdaftar dalam sistem.
5. Add Meeting Room
Menambahkan ruang meeting baru ke dalam sistem, mencakup meetingroom_id, meetingroom_name, meetingroom_capacity, dan fasilitas yang tersedia.
6. Edit Meeting Room
Memungkinkan pengguna untuk mengedit data ruang meeting yang sudah ada, seperti nama, kapasitas, dan fasilitas.
7. Delete Meeting Room
Menghapus ruang meeting yang terdaftar berdasarkan meetingroom_id.
8. Book Meeting Room
Mengizinkan karyawan untuk memesan ruang meeting untuk waktu tertentu dengan mencatat informasi terkait pemesanan seperti employee_id, meetingroom_id, booking_date, booking_time, dan occasion.
9. Add Seat
Menambahkan kursi baru ke dalam sistem dengan data seperti seat_id, seat_name, dan seat_location.
10. Edit Seat

Memungkinkan pengguna untuk mengedit data kursi yang sudah ada, seperti nama dan lokasi.

11. Delete Seat

Menghapus kursi yang terdaftar dalam sistem berdasarkan seat_id.

12. Book Seat

Memungkinkan karyawan untuk memesan kursi untuk waktu tertentu, mencatat employee_id, seat_id, booking_date, booking_time, dan booking_end.

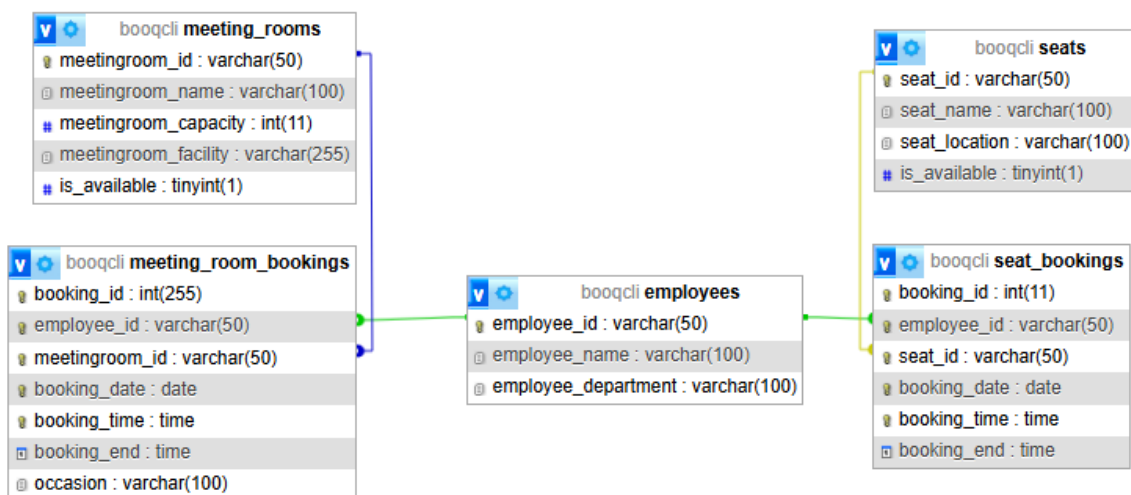
13. Laporan Penggunaan (Usage Report)

Menyediakan laporan yang merinci penggunaan ruang meeting dan kursi berdasarkan periode tertentu (misalnya, bulanan atau tahunan).

14. Exit

Menghentikan aplikasi

3.4 Database



Gambar 3. 6 Database

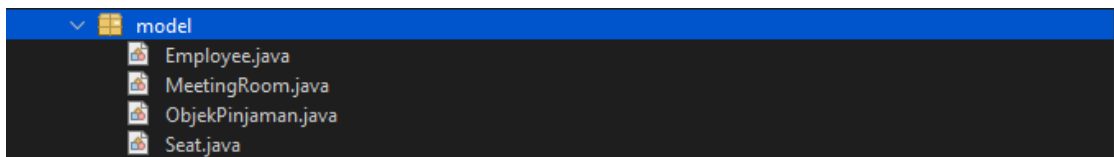
Aplikasi ini menggunakan MySQL sebagai database untuk menyimpan data terkait karyawan, ruang meeting, kursi, dan pemesanan. Tabel-tabel utama dalam database ini meliputi:

1. employees: Menyimpan data karyawan yang terdaftar dalam sistem.
2. meeting_rooms: Menyimpan data ruang meeting yang tersedia.
3. seats: Menyimpan data kursi yang tersedia.
4. meeting_room_bookings: Menyimpan data pemesanan ruang meeting.
5. seat_bookings: Menyimpan data pemesanan kursi.

3.5 OOP Implementation

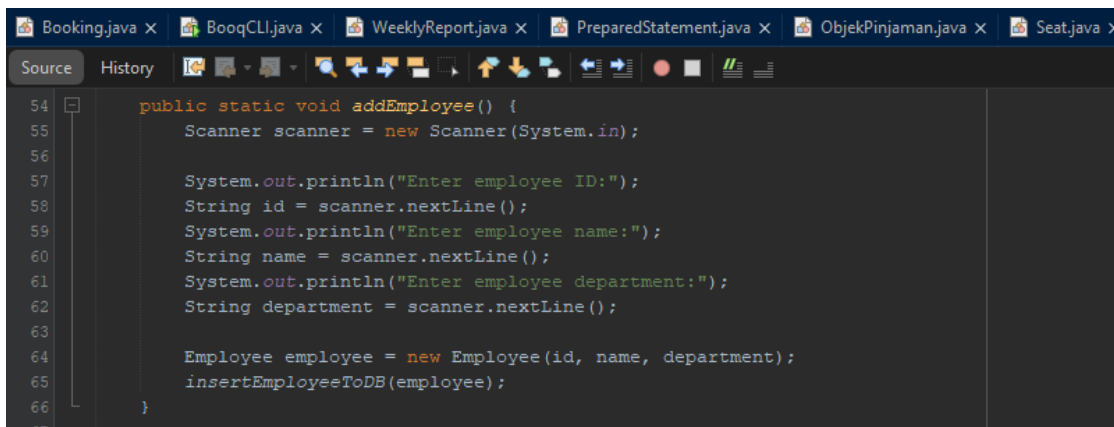
1. Class, Object, dan Package

Kelas Employee, MeetingRoom, dan Seat adalah contoh kelas yang ada dalam aplikasi kami. Setiap kelas ini menggambarkan entitas yang berbeda dalam sistem manajemen pemesanan ruang rapat dan kursi. Kelas Employee mendefinisikan atribut dan perilaku yang terkait dengan karyawan, seperti ID dan nama karyawan. Kelas MeetingRoom mendefinisikan atribut ruang rapat, seperti kapasitas dan fasilitas yang tersedia, sedangkan Seat mendefinisikan atribut kursi seperti lokasi dan status ketersediaan.



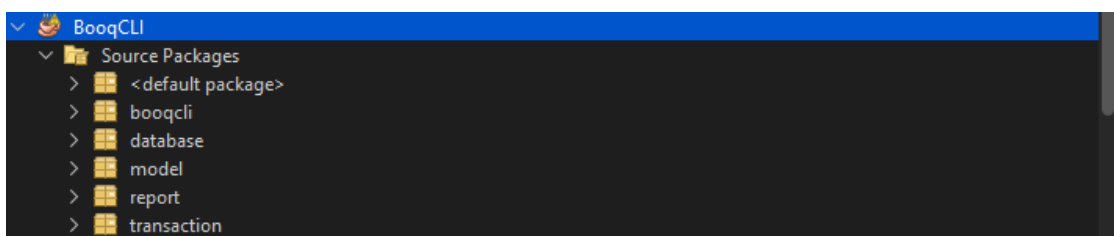
Gambar 3. 7 Class

Objek adalah instansi nyata dari kelas yang didefinisikan. Sebagai contoh, ketika membuat object scanner dan object employee.



Gambar 3. 8 Membuat Object Scanner dan Employee

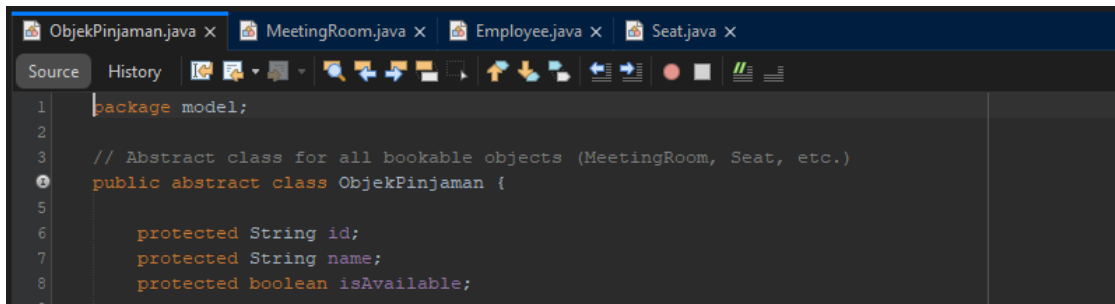
Package adalah cara untuk mengorganisir kelas-kelas dalam proyek sehingga lebih terstruktur dan mudah dipelihara. Dalam proyek ini, kami menggunakan beberapa package, seperti dibawah ini.



Gambar 3. 9 Package

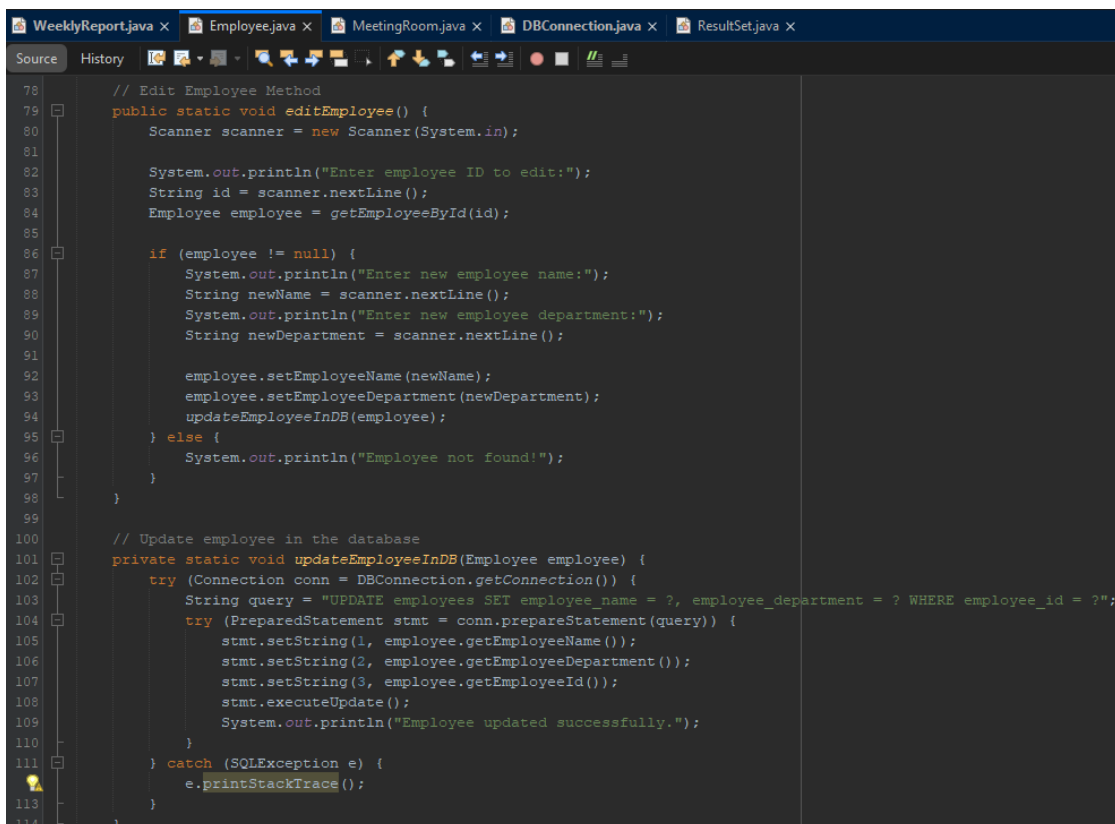
2. Variable dan methods

Variabel digunakan untuk menyimpan data atau informasi terkait objek yang dikelola dalam aplikasi. Setiap kelas memiliki variabel yang sesuai dengan entitas yang diwakili. Sebagai contoh, kelas `Employee` memiliki variabel `employeeId`, `employeeName`, dan `employeeDepartment`, yang menyimpan informasi tentang karyawan. Kelas `MeetingRoom` memiliki variabel `meetingroomId`, `meetingroomName`, `meetingroomCapacity`, dan `meetingroomFacility`, yang menyimpan data tentang ruang rapat.



Gambar 3. 10 Variable ObjekPinjaman

Method adalah fungsi yang didefinisikan di dalam kelas yang memungkinkan kita untuk melakukan operasi pada objek atau mendapatkan informasi dari objek tersebut. Method dalam aplikasi ini mencakup tindakan-tindakan yang dilakukan terhadap objek, seperti `addEmployee()`, `editEmployee()`, `deleteEmployee()`, dan `seeAllEmployees()`.

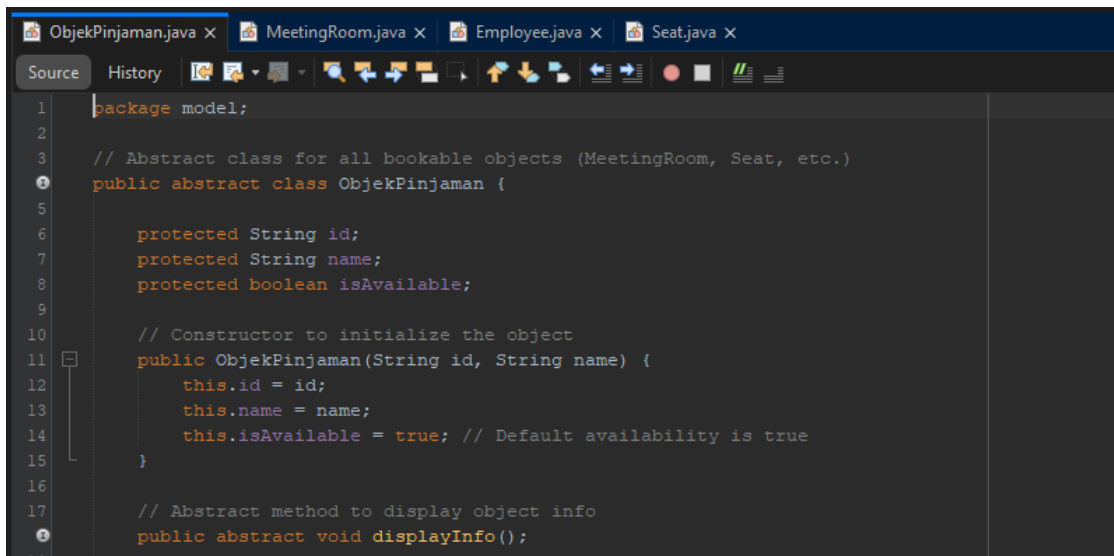


Gambar 3. 11 Method Employee

3. Inheritance

Class ObjekPinjaman adalah kelas abstrak yang memiliki atribut umum yang digunakan oleh kelas Employee, MeetingRoom, dan Seat. Kelas-kelas tersebut mewarisi sifat dan perilaku dari ObjekPinjaman, seperti atribut id dan name, yang menunjukkan bahwa kelas-kelas tersebut mewarisi karakteristik dari kelas dasar ObjekPinjaman.

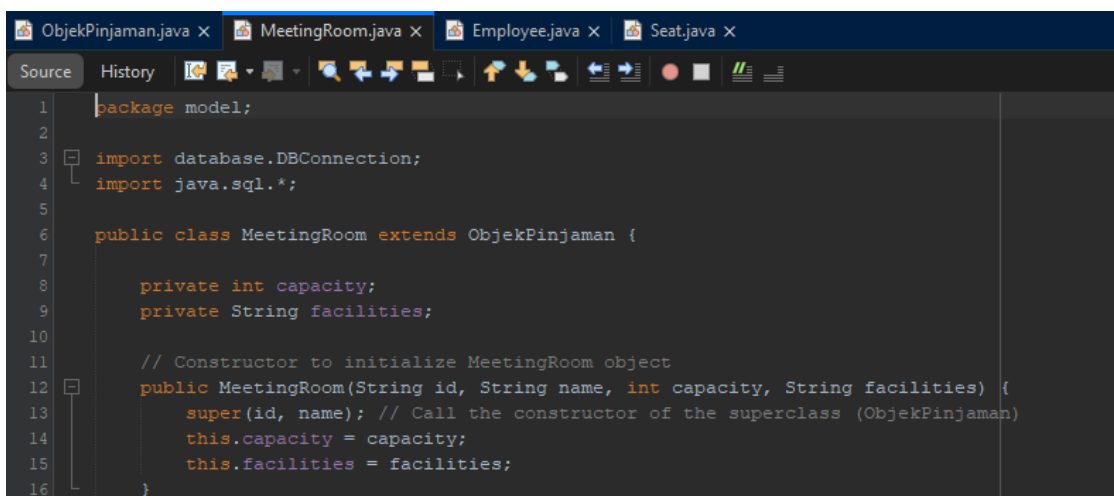
Class ObjectPinjaman:



```
1 package model;
2
3 // Abstract class for all bookable objects (MeetingRoom, Seat, etc.)
4 public abstract class ObjekPinjaman {
5
6     protected String id;
7     protected String name;
8     protected boolean isAvailable;
9
10    // Constructor to initialize the object
11    public ObjekPinjaman(String id, String name) {
12        this.id = id;
13        this.name = name;
14        this.isAvailable = true; // Default availability is true
15    }
16
17    // Abstract method to display object info
18    public abstract void displayInfo();
19 }
```

Gambar 3. 12 ObjectPinjaman Class

Class MeetingRoom yang mewarisi:



```
1 package model;
2
3 import database.DBConnection;
4 import java.sql.*;
5
6 public class MeetingRoom extends ObjekPinjaman {
7
8     private int capacity;
9     private String facilities;
10
11    // Constructor to initialize MeetingRoom object
12    public MeetingRoom(String id, String name, int capacity, String facilities) {
13        super(id, name); // Call the constructor of the superclass (ObjekPinjaman)
14        this.capacity = capacity;
15        this.facilities = facilities;
16    }
17 }
```

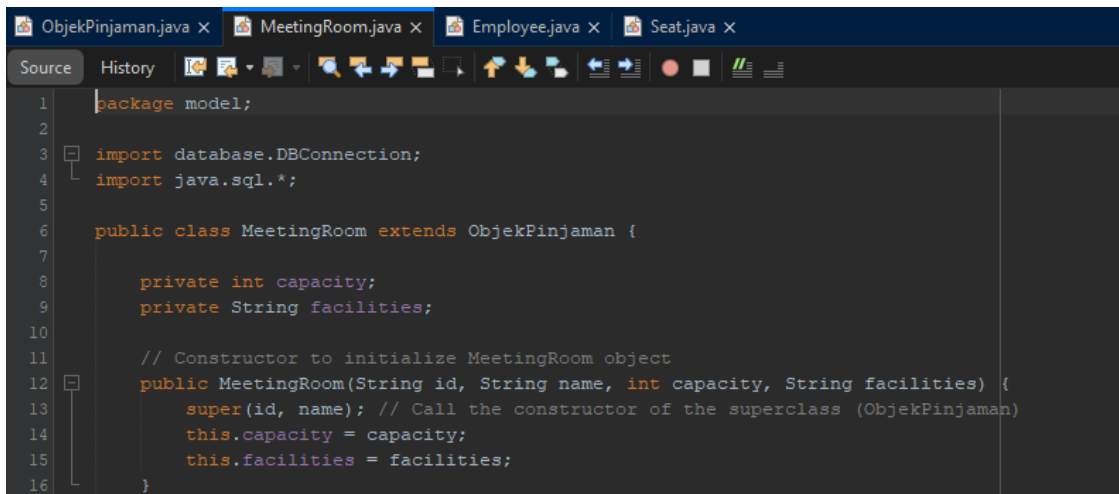
Gambar 3. 13 MeetingRoom Class Inherit ObjectPinjaman

4. Encapsulation

Encapsulation diterapkan dengan menggunakan access modifiers

Ada empat jenis access modifiers di Java, yaitu:

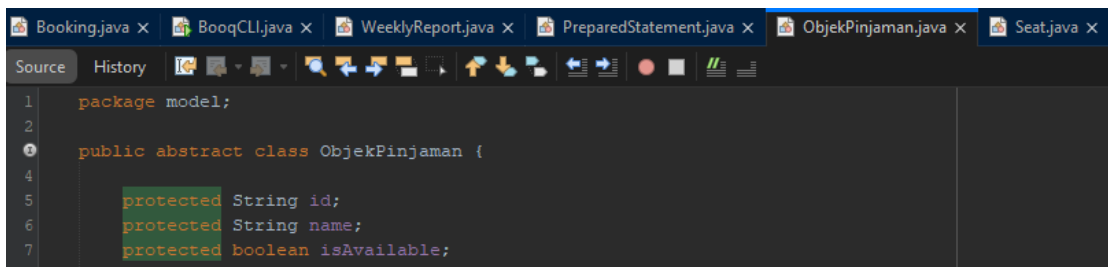
public: Akses tanpa batas. Elemen yang diberi modifier ini dapat diakses dari kelas manapun, baik dalam satu paket (package) maupun luar paket.



```
1 package model;
2
3 import database.DBConnection;
4 import java.sql.*;
5
6 public class MeetingRoom extends ObjekPinjaman {
7
8     private int capacity;
9     private String facilities;
10
11     // Constructor to initialize MeetingRoom object
12     public MeetingRoom(String id, String name, int capacity, String facilities) {
13         super(id, name); // Call the constructor of the superclass (ObjekPinjaman)
14         this.capacity = capacity;
15         this.facilities = facilities;
16     }
17 }
```

Gambar 3. 14 Public Class

protected: Akses dibatasi untuk kelas itu sendiri, kelas dalam paket yang sama, dan subclass (kelas turunan).



```
1 package model;
2
3 public abstract class ObjekPinjaman {
4
5     protected String id;
6     protected String name;
7     protected boolean isAvailable;
8 }
```

Gambar 3. 15 Protected Variable

private: Hanya dapat diakses dalam kelas yang mendeklarasikannya. Elemen yang diberi modifier ini tidak dapat diakses oleh kelas lain, termasuk kelas yang berada dalam paket yang sama. Misal, pada kelas Employee, properti seperti employeeId, employeeName, dan employeeDepartment disembunyikan dari akses langsung oleh pengguna kode lainnya, dan akses terhadapnya diberikan melalui metode getter dan setter.

```
10 public class Employee {
11     private String employeeId;
12     private String employeeName;
13     private String employeeDepartment;
14
15     // Constructor
16     public Employee(String employeeId, String employeeName, String employeeDepartment) {
17         this.employeeId = employeeId;
18         this.employeeName = employeeName;
19         this.employeeDepartment = employeeDepartment;
20     }
21
22     // Getters and Setters
23     public String getEmployeeId() {
24         return employeeId;
25     }
26
27     public void setEmployeeId(String employeeId) {
28         this.employeeId = employeeId;
29     }
30
31     public String getEmployeeName() {
32         return employeeName;
33     }
34
35     public void setEmployeeName(String employeeName) {
36         this.employeeName = employeeName;
37     }
38
39     public String getEmployeeDepartment() {
40         return employeeDepartment;
41     }
42
43     public void setEmployeeDepartment(String employeeDepartment) {
44         this.employeeDepartment = employeeDepartment;
45     }
}
```

Gambar 3. 16 Encapsulation Employee Class

5. Polymorphism

Polymorphism digunakan ketika kita ingin menggunakan metode yang sama dengan perilaku berbeda tergantung pada objek yang memanggilnya. Dalam kasus ini, kita menggunakan metode `displayInfo()` yang diimplementasikan secara berbeda pada Smasing-masing objek, seperti `Employee`, `MeetingRoom`, dan `Seat`.

Overriding:

Abstract method `displayInfo()`:

```

ObjekPinjaman.java x MeetingRoom.java x Employee.java x Seat.java x
Source History
public abstract class ObjekPinjaman {
    5
    6     protected String id;
    7     protected String name;
    8     protected boolean isAvailable;
    9
    10    // Constructor to initialize the object
    11    public ObjekPinjaman(String id, String name) {
    12        this.id = id;
    13        this.name = name;
    14        this.isAvailable = true; // Default availability is true
    15    }
    16
    17    // Abstract method to display object info
    18    public abstract void displayInfo();
    19
}

```

Gambar 3. 17 Polymorphism displayInfo()

MeetingRoom displayInfo():

```

ObjekPinjaman.java x MeetingRoom.java x Employee.java x Seat.java x
Source History
    36    // Display Meeting Room information
    37    @Override
    38    public void displayInfo() {
    39        System.out.println("Meeting Room ID: " + id);
    40        System.out.println("Name: " + name);
    41        System.out.println("Capacity: " + capacity);
    42        System.out.println("Facilities: " + facilities);
    43        System.out.println("Available: " + (isAvailable ? "Yes" : "No"));
    44    }

```

Gambar 3. 18 Overriding displayInfo() Room

Seat displayInfo()

```

ObjekPinjaman.java x MeetingRoom.java x Employee.java x Seat.java x
Source History
    26    // Display Seat information
    27    @Override
    28    public void displayInfo() {
    29        System.out.println("Seat ID: " + id);
    30        System.out.println("Name: " + name);
    31        System.out.println("Location: " + location);
    32        System.out.println("Available: " + (isAvailable ? "Yes" : "No"));
    33    }

```

Gambar 3. 19 Overriding displayInfo() Seat

Overloading:

```

Booking.java x BooqCLI.java x WeeklyReport.java x PreparedStatement.java x ObjekPinjaman.java x Seat.java x E
Source History
    16    public Employee(String employeeId, String employeeName, String employeeDepartment) {
    17        this.employeeId = employeeId;
    18        this.employeeName = employeeName;
    19        this.employeeDepartment = employeeDepartment;
    20    }
    21
    22    public Employee(String employeeId, String employeeNam) {
    23        this.employeeId = employeeId;
    24        this.employeeName = employeeName;
    25        this.employeeDepartment = "Unknown";
    26    }

```

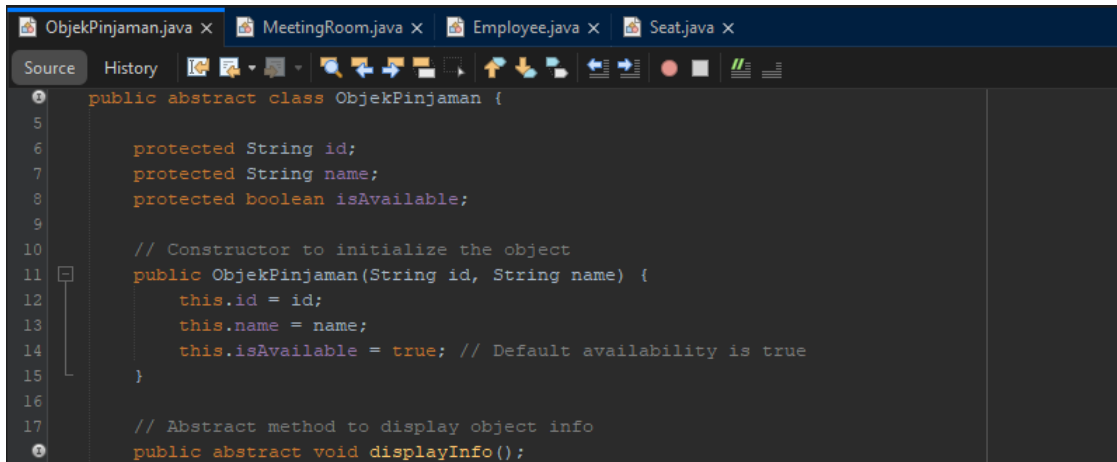
Gambar 3. 20 Overriding

6. Abstraction

Abstraction diterapkan dengan menggunakan abstract class dan interface. Pada kelas Booking dan Report, kami menggunakan kelas abstrak dan interface untuk mendefinisikan kerangka dasar yang akan diimplementasikan oleh kelas-kelas lain.

Abstract class:

Abstract class ObjekPinjaman yang memiliki abstract method displayInfo() yang nanti di override di MeetingRoom, Seat, Employee:

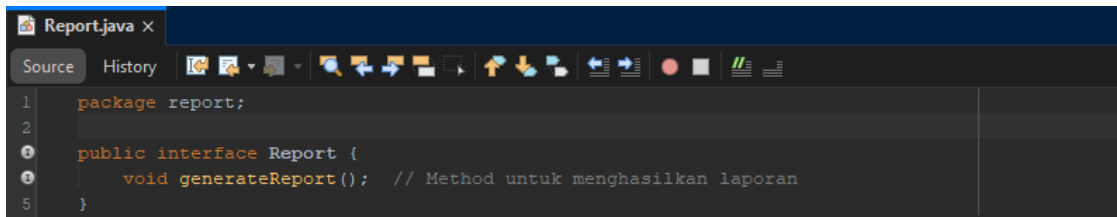


```
ObjekPinjaman.java x MeetingRoom.java x Employee.java x Seat.java x
Source History
1 public abstract class ObjekPinjaman {
2
3     protected String id;
4     protected String name;
5     protected boolean isAvailable;
6
7     // Constructor to initialize the object
8     public ObjekPinjaman(String id, String name) {
9         this.id = id;
10        this.name = name;
11        this.isAvailable = true; // Default availability is true
12    }
13
14    // Abstract method to display object info
15    public abstract void displayInfo();
16
17 }
```

Gambar 3. 21 Abstract class

Interface

Interface report yang miliki method generateReport.



```
Report.java x
Source History
1 package report;
2
3 public interface Report {
4     void generateReport(); // Method untuk menghasilkan laporan
5 }
```

Gambar 3. 22 Interface report

Kelas yang mengimplementasikannya:

```

22 public class WeeklyReport implements Report {
23
24     private int week; // Menyimpan minggu yang akan dilaporkan
25     private int year; // Menyimpan tahun yang akan dilaporkan
26
27     public WeeklyReport(int week) {
28         this.week = week;
29         this.year = year;
30     }
31
32     @Override
33     public void generateReport() {
34         try (Connection conn = DBConnection.getConnection()) {
35             String query = "SELECT * FROM meeting_room_bookings WHERE WEEK(booking_date, 1) = ? AND YEAR(booking_date) = ?";
36             try (PreparedStatement stmt = conn.prepareStatement(query)) {
37                 stmt.setInt(1, week);
38                 stmt.setInt(2, year);
39                 ResultSet rs = stmt.executeQuery();
40
41                 while (rs.next()) {
42                     System.out.println("Booking ID: " + rs.getInt("booking_id"));
43                     System.out.println("Employee ID: " + rs.getString("employee_id"));
44                     System.out.println("Object ID: " + rs.getString("object_id"));
45                     System.out.println("Booking Date: " + rs.getString("booking_date"));
46                     System.out.println("Booking Time: " + rs.getString("booking_time"));
47                     System.out.println("End Time: " + rs.getString("booking_end"));
48                     System.out.println("-----");
49                 }
50             }
51         } catch (SQLException e) {
52             e.printStackTrace();
53         }
54     }
55 }

```

Gambar 3. 23 WeeklyReport Class

```

12 public class MonthlyReport implements Report {
13
14     private int month; // Menyimpan bulan yang akan dilaporkan
15     private int year; // Menyimpan tahun yang akan dilaporkan
16
17     public MonthlyReport(int month, int year) {
18         this.month = month;
19         this.year = year;
20     }
21
22     @Override
23     public void generateReport() {
24         try (Connection conn = DBConnection.getConnection()) {
25             // Query untuk mengambil data booking berdasarkan bulan dan tahun
26             String query = "SELECT * FROM meeting_room_bookings WHERE MONTH(booking_date) = ? AND YEAR(booking_date) = ?";
27             try (PreparedStatement stmt = conn.prepareStatement(query)) {
28                 stmt.setInt(1, month); // Set bulan
29                 stmt.setInt(2, year); // Set tahun
30                 ResultSet rs = stmt.executeQuery();
31
32                 // Menampilkan hasil laporan
33                 while (rs.next()) {
34                     System.out.println("Booking ID: " + rs.getInt("booking_id"));
35                     System.out.println("Employee ID: " + rs.getString("employee_id"));
36                     System.out.println("Meeting Room ID: " + rs.getString("meetingroom_id"));
37                     System.out.println("Booking Date: " + rs.getString("booking_date"));
38                     System.out.println("Booking Time: " + rs.getString("booking_time"));
39                     System.out.println("End Time: " + rs.getString("booking_end"));
40                     System.out.println("Occasion: " + rs.getString("occasion"));
41                     System.out.println("-----");
42                 }
43             }
44         } catch (SQLException e) {
45             e.printStackTrace();
46         }
47     }
48 }

```

Gambar 3. 24 MonthlyReport Class

```

9 public class YearlyReport implements Report {
10
11     private int year; // Menyimpan tahun yang akan dilaporkan
12
13     public YearlyReport(int year) {
14         this.year = year;
15     }
16
17     @Override
18     public void generateReport() {
19         try (Connection conn = DBConnection.getConnection()) {
20             // Query untuk mengambil data booking berdasarkan tahun
21             String query = "SELECT * FROM meeting_room_bookings WHERE YEAR(booking_date) = ?";
22             try (PreparedStatement stmt = conn.prepareStatement(query)) {
23                 stmt.setInt(1, year); // Set tahun
24                 ResultSet rs = stmt.executeQuery();
25
26                 // Menampilkan hasil laporan
27                 while (rs.next()) {
28                     System.out.println("Booking ID: " + rs.getInt("booking_id"));
29                     System.out.println("Employee ID: " + rs.getString("employee_id"));
30                     System.out.println("Meeting Room ID: " + rs.getString("meetingroom_id"));
31                     System.out.println("Booking Date: " + rs.getString("booking_date"));
32                     System.out.println("Booking Time: " + rs.getString("booking_time"));
33                     System.out.println("End Time: " + rs.getString("booking_end"));
34                     System.out.println("Occasion: " + rs.getString("occasion"));
35                     System.out.println("-----");
36                 }
37             }
38         } catch (SQLException e) {
39             e.printStackTrace();
40         }
41     }
42 }

```

Gambar 3. 25 YearlyReport

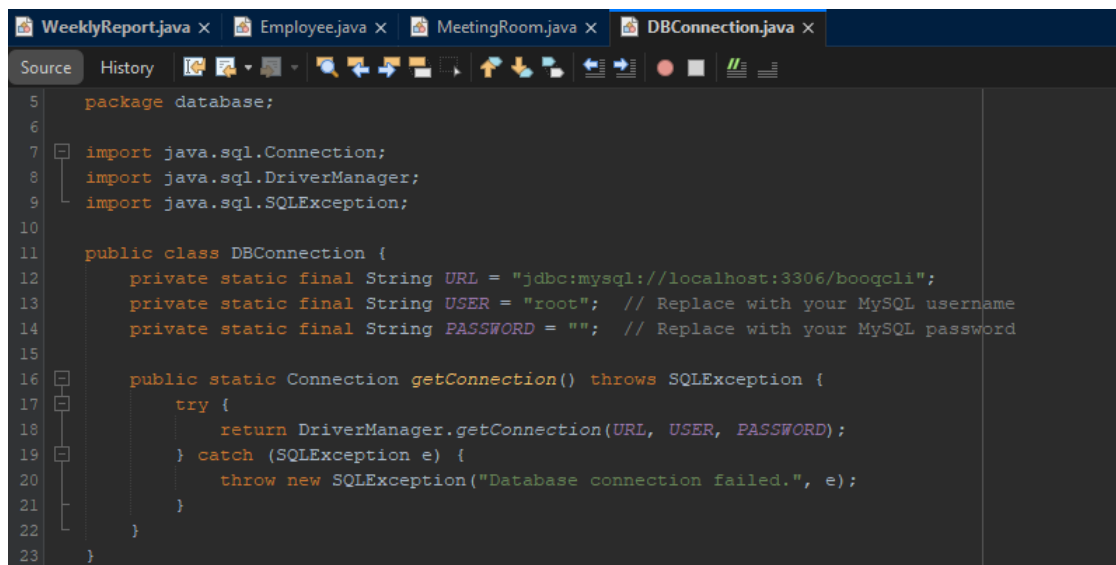
```

1 package model;
2
3
4 public abstract class ObjekPinjaman {
5     protected String id;
6     protected String name;
7     protected boolean isAvailable;
8 }

```

7. Database Management

DBConnection berfungsi untuk menghubungkan aplikasi dengan database. Koneksi ini digunakan di kelas Booking, Report, dan kelas lainnya untuk menjalankan query SQL untuk operasi CRUD (Create, Read, Update, Delete).



```
5 package database;
6
7 import java.sql.Connection;
8 import java.sql.DriverManager;
9 import java.sql.SQLException;
10
11 public class DBConnection {
12     private static final String URL = "jdbc:mysql://localhost:3306/booqcli";
13     private static final String USER = "root"; // Replace with your MySQL username
14     private static final String PASSWORD = ""; // Replace with your MySQL password
15
16     public static Connection getConnection() throws SQLException {
17         try {
18             return DriverManager.getConnection(URL, USER, PASSWORD);
19         } catch (SQLException e) {
20             throw new SQLException("Database connection failed.", e);
21         }
22     }
23 }
```

Gambar 3. 26 DBConnection

BAB IV

KESIMPULAN

4.1 Ringkasan Proyek

Proyek ini bertujuan untuk mengembangkan aplikasi pemesanan ruang rapat dan kursi menggunakan prinsip-prinsip pemrograman berorientasi objek (OOP) dalam bahasa Java. Aplikasi ini dikembangkan menggunakan berbagai konsep OOP, termasuk inheritance, polymorphism, encapsulation, abstraction, dan penggunaan interfaces. Aplikasi ini juga memanfaatkan database MySQL untuk menyimpan data terkait pemesanan ruang rapat, kursi, serta data karyawan.

Struktur aplikasi ini terdiri dari beberapa bagian utama, termasuk:

1. Model: Menyimpan kelas-kelas yang mewakili entitas utama, seperti Employee, MeetingRoom, dan Seat.
2. Transaksi: Menangani logika pemesanan, yaitu Booking yang digunakan untuk membuat pemesanan ruang rapat atau kursi.
3. Report: Menyediakan fungsionalitas untuk menghasilkan laporan pemesanan, baik per minggu maupun per bulan.
4. Database: Koneksi ke database MySQL yang menyimpan data terkait entitas dan transaksi.

4.2 Hasil yang Dicapai

Proyek ini berhasil mencapai tujuan untuk mengimplementasikan aplikasi pemesanan ruang rapat dan kursi dengan menggunakan prinsip-prinsip OOP. Hasil utama yang dicapai meliputi:

1. Penggunaan OOP: Semua bagian dari aplikasi ini memanfaatkan prinsip OOP, seperti inheritance yang digunakan untuk mendefinisikan hierarki antara kelas, polymorphism yang memungkinkan penggunaan metode yang sama pada objek yang berbeda, dan encapsulation yang menjaga akses ke data hanya melalui metode yang tepat.
2. Modularitas Kode: Kode dipisahkan menjadi beberapa paket (package) untuk memisahkan antara model, transaksi, dan laporan, yang membuat aplikasi lebih mudah dipelihara dan dikembangkan.
3. Database Integration: Aplikasi terhubung dengan database MySQL untuk menyimpan dan mengambil data mengenai karyawan, ruang rapat, kursi, dan pemesanan.

4. Laporan: Fitur laporan yang mengizinkan pengguna untuk melihat pemesanan dalam periode tertentu (mingguan atau bulanan) sesuai dengan permintaan.

4.3 Kesimpulan Akhir

Secara keseluruhan, proyek ini berhasil membangun aplikasi pemesanan ruang rapat dan kursi yang efektif dan efisien menggunakan prinsip OOP. Implementasi inheritance, polymorphism, encapsulation, dan abstraction telah diterapkan dengan baik, sehingga aplikasi menjadi fleksibel dan mudah dipelihara. Aplikasi ini juga dapat menangani berbagai transaksi pemesanan serta menyediakan laporan yang berguna untuk menganalisis penggunaan ruang dan kursi.